

result_exp

September 18, 2026

```
[6]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# =====
# LOAD DATA
# =====

csv_path = "/home/mapa/Documents/Tec/NextGenS/FacesDetection/
↳CV-FaceMorphs-master/dev/FaceMorphing/ControlledMorphGeneration/
↳MorphGeneration_Pipeline_DistributionalSimilarity/TrustRegions/results_mixed/
↳cluster_quality_summary.csv"

df = pd.read_csv(csv_path)

print("\n[Cluster Quality Summary]")
print(df)
```

```
[Cluster Quality Summary]
   cluster_id  n_samples  original_n  clean_n  clean_retention_rate \
0           393         918        1561      918             0.588085
1            92        1186        1310     1186             0.905344
2           118         717         973      717             0.736896
3            40         835         967      835             0.863495
4           280         405         926      405             0.437365
5           460          46          59       46             0.779661
6           144          21          59       21             0.355932
7            30          51          58       51             0.879310
8           448          40          58       40             0.689655
9             0           8          60        8             0.133333
10          261          16          20       16             0.800000
11          363          13          20       13             0.650000
12          485           9          20        9             0.450000
13           17          15          20       15             0.750000
14           69           6          20        6             0.300000
```

	compression_mean_radius	compression_std_radius \
0	16.832073	1.879307
1	16.858506	1.814228
2	15.053974	1.834518
3	18.696047	1.505031
4	18.051848	1.885553
5	16.613106	2.291530
6	16.056474	2.055640
7	18.625292	1.586729
8	17.051616	2.217675
9	10.496873	1.971369
10	17.619834	1.860426
11	16.371216	2.651616
12	10.366849	2.030127
13	14.400945	4.612868
14	15.528151	1.719558

	compression_median_radius	compression_q90_radius \
0	16.816939	19.212451
1	16.749597	19.159215
2	14.958974	17.446552
3	18.594148	20.617722
4	18.007578	20.423216
5	15.941309	19.445733
6	16.312612	18.897958
7	18.848478	20.588902
8	17.038538	20.237912
9	10.128217	13.190770
10	17.624514	19.410231
11	15.912310	19.585729
12	9.696072	12.999490
13	15.133931	20.189167
14	15.066811	17.477188

	density_mean_knn_distance	density_median_knn_distance	density_knn \
0	0.230415	0.187387	4.339998
1	0.204101	0.167692	4.899531
2	0.206300	0.167486	4.847298
3	0.180786	0.141996	5.531400
4	0.304430	0.247311	3.284831
5	0.256750	0.187865	3.894843
6	0.395909	0.281591	2.525833
7	0.185710	0.114319	5.384742
8	0.191872	0.158246	5.211816
9	0.020174	0.021914	49.569400
10	0.270755	0.145804	3.693369
11	0.415751	0.392353	2.405283
12	0.447821	0.393282	2.233034

13	0.176865	0.070183	5.654043
14	1.034620	1.016656	0.966539

	fitted_distribution	ks	aic	bootstrap_model_stability
0	Gamma	0.027852	-989.667357	0.45
1	Weibull	0.016920	-1535.774965	0.84
2	Weibull	0.032209	-964.393585	0.71
3	Weibull	0.030714	-1240.739737	0.40
4	Weibull	0.024517	-219.742934	0.63
5	Lognormal	0.087685	-32.727851	0.42
6	Weibull	0.150021	7.673209	0.32
7	Lognormal	0.079159	-62.368929	0.36
8	Lognormal	0.123123	-44.486770	0.53
9	Gamma	0.180491	-42.998397	0.49
10	Weibull	0.250300	-47.920055	0.48
11	Weibull	0.172367	-32.366278	0.54
12	Weibull	0.285324	-20.870230	0.45
13	Chi-Square	0.134101	-81.895394	0.40
14	Gamma	0.225397	14.892787	0.40

```
[7]: print("\n[Dataset information]")
      print(df.info())

      print("\n[Descriptive statistics]")
      print(df.describe())
```

```
[Dataset information]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   cluster_id                            15 non-null     int64
1   n_samples                             15 non-null     int64
2   original_n                            15 non-null     int64
3   clean_n                               15 non-null     int64
4   clean_retention_rate                   15 non-null     float64
5   compression_mean_radius                15 non-null     float64
6   compression_std_radius                 15 non-null     float64
7   compression_median_radius              15 non-null     float64
8   compression_q90_radius                 15 non-null     float64
9   density_mean_knn_distance              15 non-null     float64
10  density_median_knn_distance             15 non-null     float64
11  density_knn                            15 non-null     float64
12  fitted_distribution                     15 non-null     object
13  ks                                       15 non-null     float64
14  aic                                       15 non-null     float64
```

```

15 bootstrap_model_stability    15 non-null    float64
dtypes: float64(11), int64(4), object(1)
memory usage: 2.0+ KB
None

```

[Descriptive statistics]

	cluster_id	n_samples	original_n	clean_n \
count	15.000000	15.000000	15.000000	15.000000
mean	213.333333	285.733333	408.733333	285.733333
std	178.940400	414.693943	560.959696	414.693943
min	0.000000	6.000000	20.000000	6.000000
25%	54.500000	14.000000	20.000000	14.000000
50%	144.000000	40.000000	59.000000	40.000000
75%	378.000000	561.000000	946.500000	561.000000
max	485.000000	1186.000000	1561.000000	1186.000000

	clean_retention_rate	compression_mean_radius	compression_std_radius \
count	15.000000	15.000000	15.000000
mean	0.621272	15.908187	2.127745
std	0.235188	2.529990	0.743907
min	0.133333	10.366849	1.505031
25%	0.443683	15.291063	1.824373
50%	0.689655	16.613106	1.885553
75%	0.789831	17.335725	2.136658
max	0.905344	18.696047	4.612868

	compression_median_radius	compression_q90_radius \
count	15.000000	15.000000
mean	15.788669	18.592149
std	2.675039	2.433325
min	9.696072	12.999490
25%	15.100371	18.187573
50%	16.312612	19.410231
75%	17.331526	20.213540
max	18.848478	20.617722

	density_mean_knn_distance	density_median_knn_distance	density_knn \
count	15.000000	15.000000	15.000000
mean	0.301484	0.246272	6.962797
std	0.230229	0.236707	11.870324
min	0.020174	0.021914	0.966539
25%	0.188791	0.143900	2.905332
50%	0.230415	0.167692	4.339998
75%	0.350169	0.264451	5.298279
max	1.034620	1.016656	49.569400

	ks	aic	bootstrap_model_stability
count	15.000000	15.000000	15.000000

mean	0.121345	-352.892433	0.494667
std	0.088450	535.015565	0.139533
min	0.016920	-1535.774965	0.320000
25%	0.031461	-592.068260	0.400000
50%	0.123123	-47.920055	0.450000
75%	0.176429	-32.547065	0.535000
max	0.285324	14.892787	0.840000

```
[8]: # Order clusters according to their original population
df = df.sort_values(
    by="original_n",
    ascending=False
).reset_index(drop=True)

# Create a categorical label for plots
df["cluster_label"] = "Cluster " + df["cluster_id"].astype(str)

display_columns = [
    "cluster_id",
    "original_n",
    "clean_n",
    "clean_retention_rate",
    "compression_mean_radius",
    "compression_std_radius",
    "compression_median_radius",
    "compression_q90_radius",
    "density_mean_knn_distance",
    "density_median_knn_distance",
    "density_knn",
    "fitted_distribution",
    "ks",
    "aic",
    "bootstrap_model_stability"
]

summary_table = df[display_columns].copy()

# Format percentages
summary_table["clean_retention_rate"] *= 100
summary_table["bootstrap_model_stability"] *= 100

print("\n[Cluster Quality Metrics Table]")
print(summary_table.to_string(index=False))
```

```
[Cluster Quality Metrics Table]
cluster_id  original_n  clean_n  clean_retention_rate  compression_mean_radius
```

compression_std_radius	compression_median_radius	compression_q90_radius	density_mean_knn_distance	density_median_knn_distance	density_knn	fitted_distribution	ks	aic	bootstrap_model_stability
393	1561	918	58.808456		16.832073				
1.879307		16.816939	19.212451						
0.230415		0.187387	4.339998		Gamma	0.027852			
-989.667357		45.0							
92	1310	1186	90.534351		16.858506				
1.814228		16.749597	19.159215						
0.204101		0.167692	4.899531		Weibull	0.016920			
-1535.774965		84.0							
118	973	717	73.689620		15.053974				
1.834518		14.958974	17.446552						
0.206300		0.167486	4.847298		Weibull	0.032209			
-964.393585		71.0							
40	967	835	86.349535		18.696047				
1.505031		18.594148	20.617722						
0.180786		0.141996	5.531400		Weibull	0.030714			
-1240.739737		40.0							
280	926	405	43.736501		18.051848				
1.885553		18.007578	20.423216						
0.304430		0.247311	3.284831		Weibull	0.024517			
-219.742934		63.0							
0	60	8	13.333333		10.496873				
1.971369		10.128217	13.190770						
0.020174		0.021914	49.569400		Gamma	0.180491			
-42.998397		49.0							
460	59	46	77.966102		16.613106				
2.291530		15.941309	19.445733						
0.256750		0.187865	3.894843		Lognormal	0.087685			
-32.727851		42.0							
144	59	21	35.593220		16.056474				
2.055640		16.312612	18.897958						
0.395909		0.281591	2.525833		Weibull	0.150021			
7.673209		32.0							
30	58	51	87.931034		18.625292				
1.586729		18.848478	20.588902						
0.185710		0.114319	5.384742		Lognormal	0.079159			
-62.368929		36.0							
448	58	40	68.965517		17.051616				
2.217675		17.038538	20.237912						
0.191872		0.158246	5.211816		Lognormal	0.123123			
-44.486770		53.0							
261	20	16	80.000000		17.619834				
1.860426		17.624514	19.410231						
0.270755		0.145804	3.693369		Weibull	0.250300			
-47.920055		48.0							
363	20	13	65.000000		16.371216				

2.651616		15.912310	19.585729	
0.415751		0.392353	2.405283	Weibull 0.172367
-32.366278		54.0		
485	20	9	45.000000	10.366849
2.030127		9.696072	12.999490	
0.447821		0.393282	2.233034	Weibull 0.285324
-20.870230		45.0		
17	20	15	75.000000	14.400945
4.612868		15.133931	20.189167	
0.176865		0.070183	5.654043	Chi-Square 0.134101
-81.895394		40.0		
69	20	6	30.000000	15.528151
1.719558		15.066811	17.477188	
1.034620		1.016656	0.966539	Gamma 0.225397
14.892787		40.0		

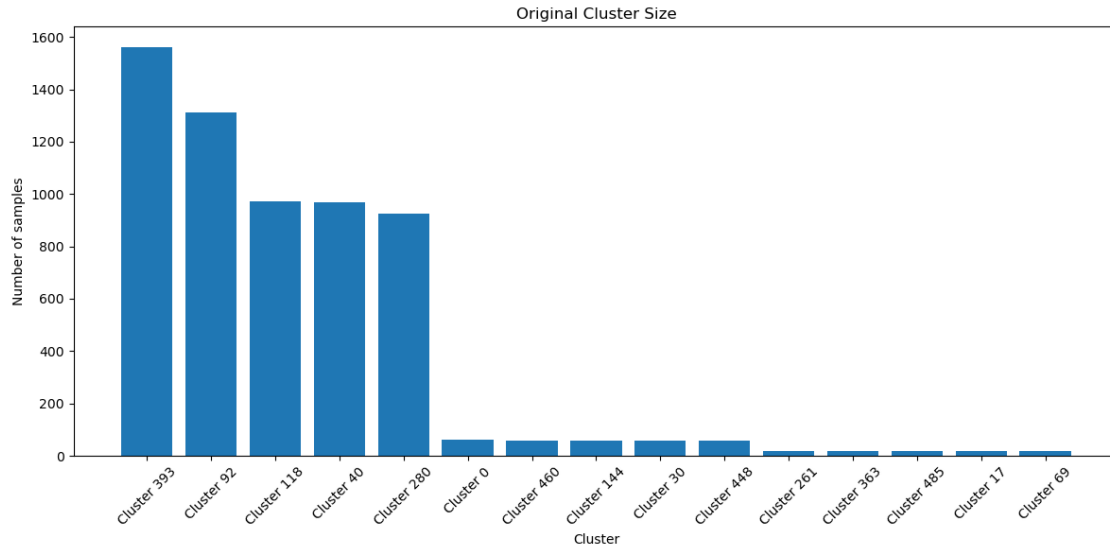
```
[9]: # =====
# 1. CLUSTER SIZE
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["original_n"]
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("Number of samples")
plt.title("Original Cluster Size")

plt.tight_layout()
plt.show()
```



```
[10]: # =====
# 2. CLEANING RETENTION
# =====

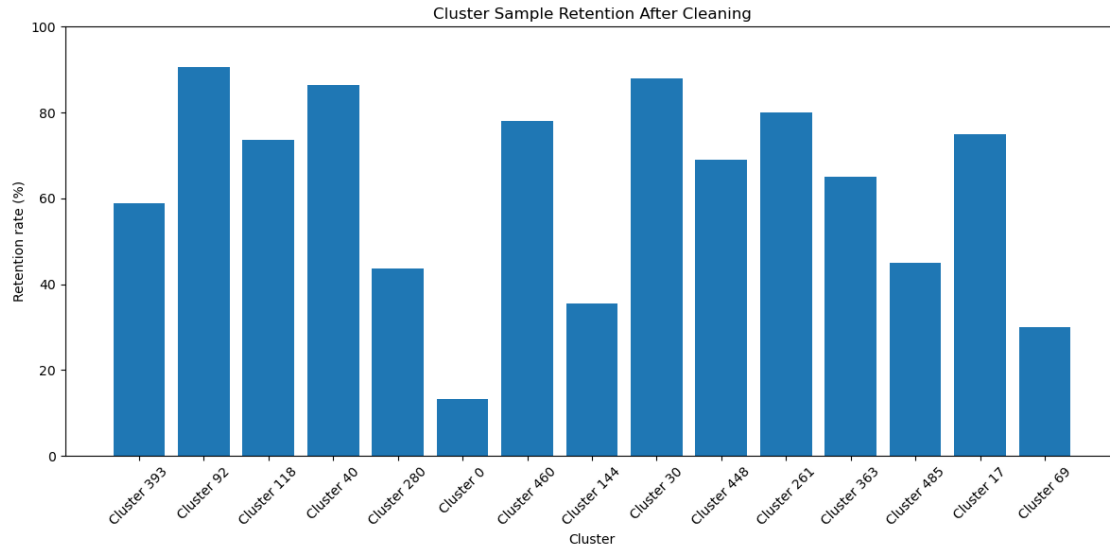
plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["clean_retention_rate"] * 100
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("Retention rate (%)")
plt.title("Cluster Sample Retention After Cleaning")

plt.ylim(0, 100)

plt.tight_layout()
plt.show()
```



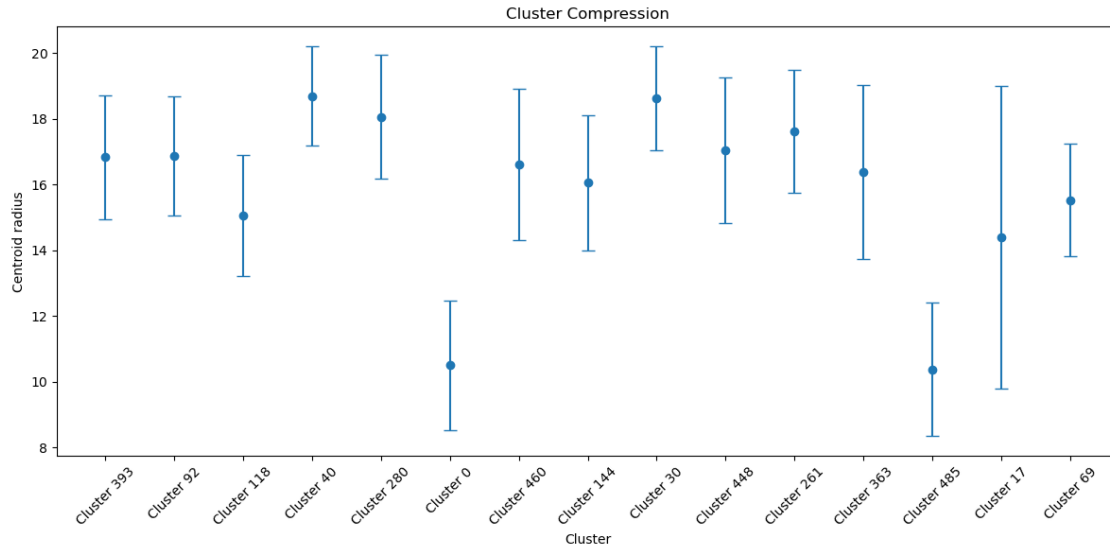
```
[11]: # =====
# 3. COMPRESSION
# =====

plt.figure(figsize=(12, 6))

plt.errorbar(
    df["cluster_label"],
    df["compression_mean_radius"],
    yerr=df["compression_std_radius"],
    fmt="o",
    capsize=5
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("Centroid radius")
plt.title("Cluster Compression")

plt.tight_layout()
plt.show()
```



```
[12]: # =====
# 4. COMPRESSION STATISTICS
# =====

compression_columns = [
    "compression_mean_radius",
    "compression_median_radius",
    "compression_q90_radius"
]

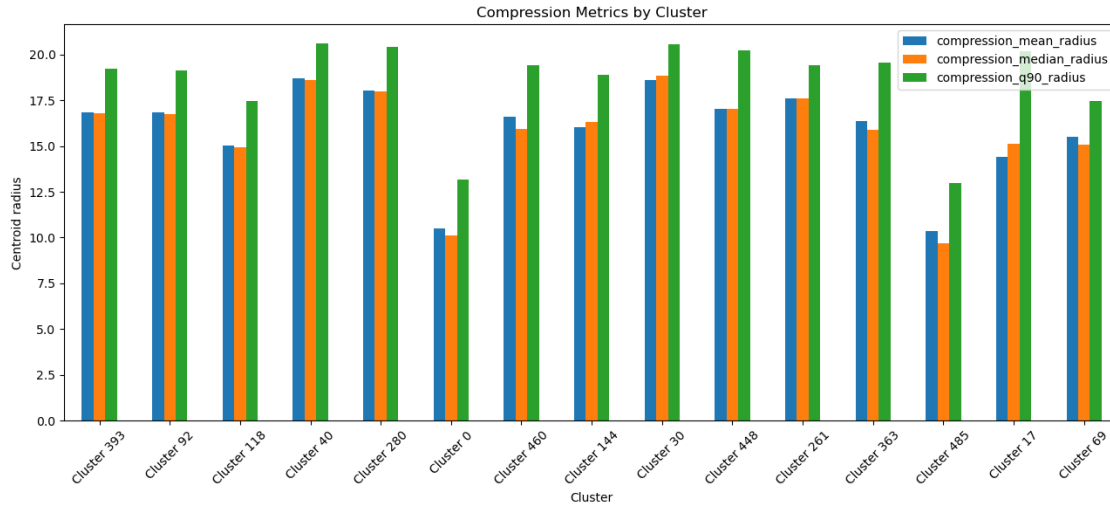
compression_plot = df[
    ["cluster_label"] + compression_columns
].set_index("cluster_label")

compression_plot.plot(
    kind="bar",
    figsize=(13, 6)
)

plt.xlabel("Cluster")
plt.ylabel("Centroid radius")
plt.title("Compression Metrics by Cluster")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



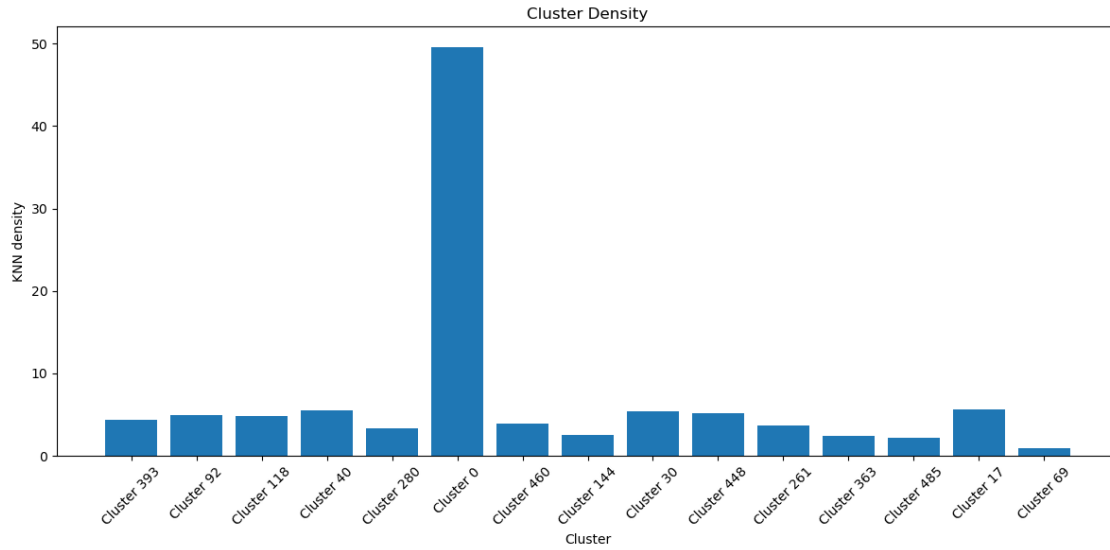
```
[13]: # =====
# 5. DENSITY
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["density_knn"]
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("KNN density")
plt.title("Cluster Density")

plt.tight_layout()
plt.show()
```



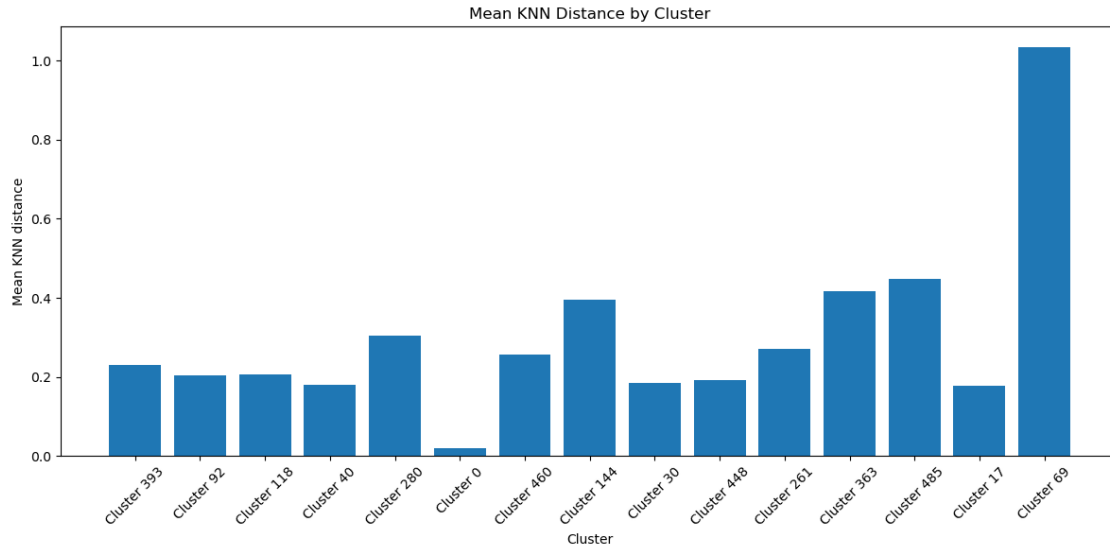
```
[14]: # =====
# 6. MEAN KNN DISTANCE
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["density_mean_knn_distance"]
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("Mean KNN distance")
plt.title("Mean KNN Distance by Cluster")

plt.tight_layout()
plt.show()
```



```
[15]: # =====
# 7. DISTRIBUTION MODEL
# =====

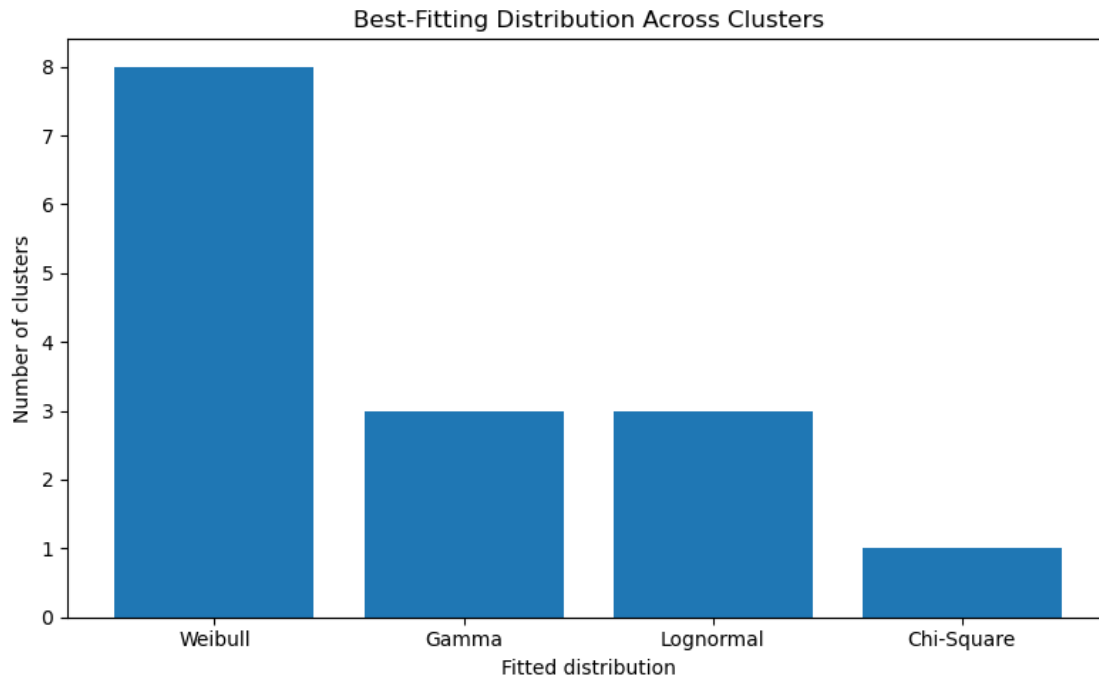
distribution_counts = (
    df["fitted_distribution"]
    .value_counts()
)

plt.figure(figsize=(8, 5))

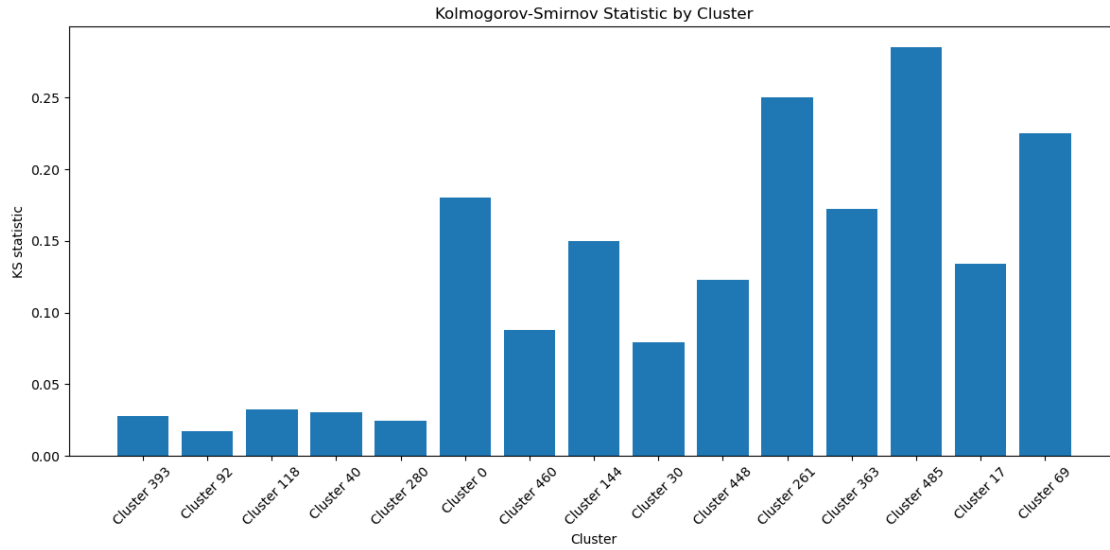
plt.bar(
    distribution_counts.index,
    distribution_counts.values
)

plt.xlabel("Fitted distribution")
plt.ylabel("Number of clusters")
plt.title("Best-Fitting Distribution Across Clusters")

plt.tight_layout()
plt.show()
```



```
[16]: # =====  
# 8. KS STATISTIC  
# =====  
  
plt.figure(figsize=(12, 6))  
  
plt.bar(  
    df["cluster_label"],  
    df["ks"]  
)  
  
plt.xticks(rotation=45)  
plt.xlabel("Cluster")  
plt.ylabel("KS statistic")  
plt.title("Kolmogorov-Smirnov Statistic by Cluster")  
  
plt.tight_layout()  
plt.show()
```



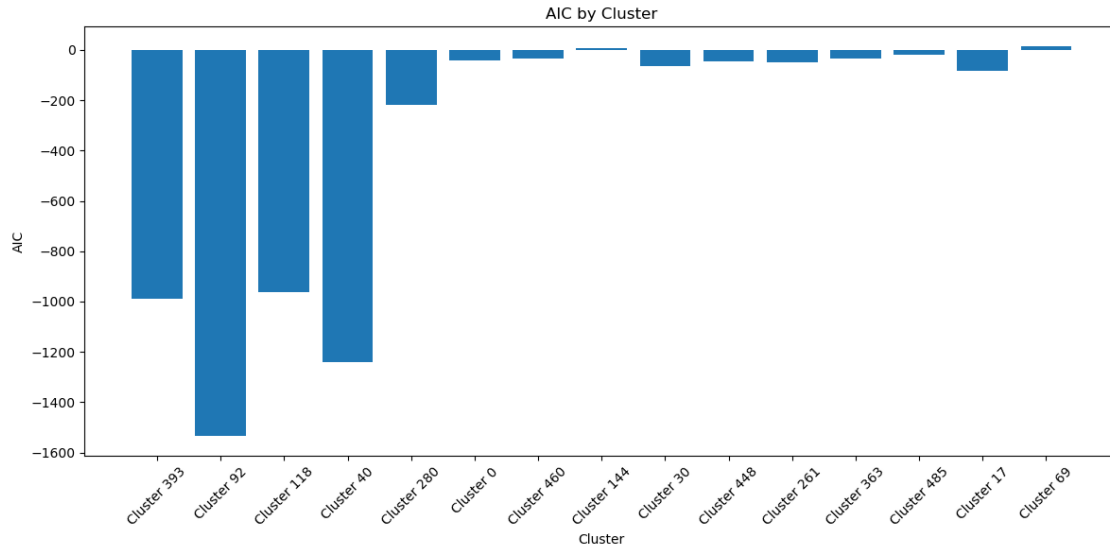
```
[17]: # =====
# 9. AIC
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["aic"]
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("AIC")
plt.title("AIC by Cluster")

plt.tight_layout()
plt.show()
```



```
[18]: # =====
# 10. BOOTSTRAP MODEL STABILITY
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    df["cluster_label"],
    df["bootstrap_model_stability"] * 100
)

plt.xticks(rotation=45)
plt.xlabel("Cluster")
plt.ylabel("Model stability (%)")
plt.title("Bootstrap Model Stability by Cluster")

plt.ylim(0, 100)

plt.tight_layout()
plt.show()
```

